

Club náutico

Realizar un programa para gestionar los datos de los usuarios de un club náutico. De cada usuario es necesario conocer su nombre, DNI y barco (se supone que cada usuario solo podrá tener un barco en el club).

De cada barco se almacena: eslora y matrícula.

El programa debe permitir:

1. Insertar un nuevo Usuario, ordenado por DNI. (No hay que ordenar sino insertar en la posición adecuada para que la lista esté en orden)
2. Mostrar todos los usuarios. Si la lista está vacía, mostrar un mensaje que lo indique
3. Buscar un Usuario por DNI o por matrícula del barco. Mostrar todos sus datos. Si no existe el usuario o el barco buscado, mostrar un mensaje que lo indique
4. Cambiar barco. Busca un usuario por DNI y modifica su barco, añadiendo los datos de matrícula y eslora del nuevo barco. Muestra el mensaje de “barco actualizado” si se encuentra el usuario o un mensaje de error en caso contrario
5. Cambiar eslora. Busca un barco por su matrícula y se cambia su eslora por la que indique el usuario. Muestra el mensaje de barco actualizado si se encuentra el usuario o un mensaje de error en caso contrario
6. Eliminar un usuario introduciendo su DNI. (Obligatorio uso de Iterator). Muestra el mensaje de usuario eliminado si existe el usuario o de error en caso contrario

Realiza un plan de pruebas para la aplicación que ya tienes creada.

Empezamos identificando las distintas funcionalidades para posteriormente identificar las regiones de equivalencia y los valores límite.

Funcionalidades a probar:

1. Insertar usuario
2. Mostrar usuarios
3. Buscar usuario por DNI
4. Buscar usuario por matricula
5. Cambiar barco de un usuario
6. Cambiar eslora de un barco
7. Eliminar usuario

Cada funcionalidad tendrá un bloque de pruebas

1. Insertar usuario (ordenado por DNI)

Clases de equivalencia

Clase de equivalencia	Descripción
CE1	Insertar al principio
CE2	Insertar en el medio
CE3	Insertar al final
CE4	Insertar un DNI ya existente
CE5	Lista vacía

Casos de prueba

Clase de equivalencia	Situación inicial	Entrada	Resultado esperado
CE1	Lista con DNIs mayores	DNI menor	Insertar al inicio
CE2	Lista con DNIs intermedios	DNI intermedio	Insertado en orden
CE3	Lista con DNIs menores	DNI mayor	Insertado al final
CE4	Usuario con DNI existente	DNI repetido	Mensaje de error
CE5	Lista vacía	Usuario	Usuario insertado

2. Mostrar usuarios

Clase de equivalencia	Descripción
CE6	Lista vacía
CE7	Lista con uno o más usuarios

Casos de prueba

Clase de equivalencia	Situación inicial	Resultado esperado
CE6	Lista vacía	Mensaje "lista vacía"
CE7	Lista con usuarios	Se muestran todos en orden

3. Buscar usuario por DNI

Clases de equivalencia

Clase de equivalencia	Descripción
CE8	DNI existente
CE9	DNI no existente
CE10	Lista vacía

Casos de prueba

Clase de equivalencia	Entrada	Resultado esperado
CE8	DNI válido existente	Se muestran los datos
CE9	DNI válido no existente	Mensaje de error
CE10	Lista vacía	Mensaje de error

4. Buscar usuario por matrícula del barco

Clases de equivalencia

Clase de equivalencia	Descripción
C11	Matrícula existente
C12	Matrícula no existente
CE13	Lista vacía

Casos de prueba

Clase de equivalencia	Entrada	Resultado esperado
CE11	Matricula existente	Se muestran los datos
CE12	Matrícula no existente	Mensaje de error
CE13	Lista vacía	Mensaje de error

5. Cambiar barco (buscar por DNI)

Clases de equivalencia

Clase de equivalencia	Descripción
C14	DNI existente
C15	DNI no existente
CE16	Lista vacía

Casos de prueba

Clase de equivalencia	Entrada	Resultado esperado
CE14	DNI existente + nuevo barco	Barco actualizado
CE15	DNI no existente	Mensaje de error
CE16	Lista vacía	Mensaje de error

6. Cambiar eslora (buscar por matrícula)

Clases de equivalencia

Clase de equivalencia	Descripción
C17	Matricula existente
C18	Matricula no existente

CE19	Lista vacía
------	-------------

Valores límite (eslora)

Valores límite	Descripción
VL1	Eslora mínima válida
VL2	Eslora negativa

Casos de prueba

Clase de equivalencia	Entrada	Resultado esperado
CE17	Matricula existente + entrada válida	Barco actualizado
CE18	Matricula existente + eslora inválida	Mensaje de error
CE19	Matricula no existente	Mensaje de error

7. Eliminar usuario (por DNI, usando Iterator)

Clases de equivalencia

Clase de equivalencia	Descripción
C20	DNI existente
C21	DNI no existente
C22	Lista vacía

Casos de prueba

Clase de equivalencia	Entrada	Resultado esperado
CE20	DNI existente	Usuario eliminado
CE21	DNI no existente	Mensaje de error
CE22	Lista vacía	Mensaje de error

JUNIT

```
// CE5: Lista vacía
```

```
@Test
```

```
void testInsertarEnListaVacia() {
```

```
    assertTrue(club.insertarUsuario(u2));
```

```
}
```

```
// CE1: Insertar al principio
```

```
@Test
```

```
void testInsertarAlPrincipio() {
```

```
    club.insertarUsuario(u2);
```

```
    club.insertarUsuario(u3);
```

```
    Usuario nuevo = new Usuario("Juan", "111A", new Barco("MAT0", 4.0));
```

```
    assertTrue(club.insertarUsuario(nuevo));
```

```
}
```

```
// CE2: Insertar en el medio
```

```
@Test
```

```
void testInsertarEnMedio() {
```

```
    club.insertarUsuario(u1);
```

```
    club.insertarUsuario(u3);
```

```
    assertTrue(club.insertarUsuario(u2));
```

```
}
```

```
// CE3: Insertar al final
```

```
@Test
```

```
void testInsertarAlFinal() {
```

```
    club.insertarUsuario(u1);
```

```
    club.insertarUsuario(u2);
```

```
    assertTrue(club.insertarUsuario(u3));
```

```
}
```

```
// CE4: DNI repetido
```

@Test

void testInsertarDniRepetido() {

club.insertarUsuario(u1);

Usuario repetido = new Usuario("Otro", "222B", new Barco("MAT9", 8.0));

assertFalse(club.insertarUsuario(repetido));

}

// CE6: Lista vacía

@Test

void testMostrarUsuariosListaVacía() {

String resultado = club.mostrarUsuarios();

assertEquals("lista vacía", resultado.toLowerCase());

}

// CE7: Lista con usuarios

@Test

void testMostrarUsuariosConDatos() {

club.insertarUsuario(u1);

club.insertarUsuario(u2);

String resultado = club.mostrarUsuarios();

assertTrue(resultado.contains("222B"));

assertTrue(resultado.contains("333C"));

}

// CE8: DNI existente

@Test

void testBuscarPorDniExistente() {

club.insertarUsuario(u1);

```
        Usuario encontrado = club.buscarUsuarioPorDni("222B");  
        assertNotNull(encontrado);  
    }  
  

```

```
// CE9: DNI no existente
```

```
@Test
```

```
void testBuscarPorDniNoExistente() {  
    club.insertarUsuario(u1);  
  

```

```
        Usuario encontrado = club.buscarUsuarioPorDni("999Z");  
        assertNull(encontrado);  
    }  
  

```

```
// CE10: Lista vacía
```

```
@Test
```

```
void testBuscarPorDniListaVacía() {  
    assertNull(club.buscarUsuarioPorDni("222B"));  
}  
  

```

```
// CE11: Matrícula existente
```

```
@Test
```

```
void testBuscarPorMatriculaExistente() {  
    club.insertarUsuario(u1);  
  

```

```
        Usuario encontrado = club.buscarUsuarioPorMatricula("MAT1");  
        assertNotNull(encontrado);  
    }  
  

```

```
// CE12: Matrícula no existente
```

```
@Test
```

```
void testBuscarPorMatriculaNoExistente() {  
  

```

```
club.insertarUsuario(u1);

assertNull(club.buscarUsuarioPorMatricula("XXX"));
}

// CE13: Lista vacía
@Test
void testBuscarPorMatriculaListaVacía() {
    assertNull(club.buscarUsuarioPorMatricula("MAT1"));
}

// CE14: DNI existente
@Test
void testCambiarBarcoDniExistente() {
    club.insertarUsuario(u1);

    Barco nuevo = new Barco("NUEVA", 8.0);
    assertTrue(club.cambiarBarco("222B", nuevo));
}

// CE15: DNI no existente
@Test
void testCambiarBarcoDniNoExistente() {
    Barco nuevo = new Barco("NUEVA", 8.0);
    assertFalse(club.cambiarBarco("999Z", nuevo));
}

// CE16: Lista vacía
@Test
void testCambiarBarcoListaVacía() {
    Barco nuevo = new Barco("NUEVA", 8.0);
```



```
        assertFalse(club.cambiarBarco("222B", nuevo));  
    }  
}
```

// CE17 + VL1: Matrícula existente + eslora válida

@Test

```
void testCambiarEsloraValida() {  
    club.insertarUsuario(u1);  
  
    assertTrue(club.cambiarEslora("MAT1", 1.0));  
}
```

// CE18 + VL2: Eslora negativa

@Test

```
void testCambiarEsloraNegativa() {  
    club.insertarUsuario(u1);  
  
    assertFalse(club.cambiarEslora("MAT1", -2.0));  
}
```

// CE19: Matrícula no existente

@Test

```
void testCambiarEsloraMatriculaNoExistente() {  
    club.insertarUsuario(u1);  
  
    assertFalse(club.cambiarEslora("XXX", 5.0));  
}
```

// CE20: DNI existente

@Test

```
void testEliminarUsuarioExistente() {  
    club.insertarUsuario(u1);
```

```
        assertTrue(club.eliminarUsuario("222B"));
    }

    // CE21: DNI no existente
    @Test
    void testEliminarUsuarioNoExistente() {
        club.insertarUsuario(u1);

        assertFalse(club.eliminarUsuario("999Z"));
    }

    // CE22: Lista vacía
    @Test
    void testEliminarUsuarioListaVacía() {
        assertFalse(club.eliminarUsuario("222B"));
    }
}
```